

C++&Qt Day03 hw3 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.07

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

이번 과제에서 Qt 를 이용하여 천지인 방식의 키보드 프로그램을 구현하는 과정에서 ChatGPT 를 사용하였다.

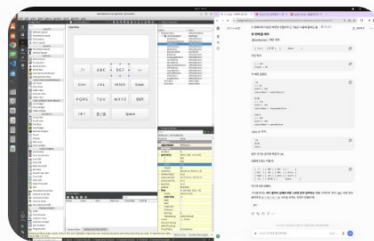
Qt 프로그램에서 여러 버튼의 동작을 연결하고, 같은 버튼을 연속해서 눌렀을 때 문자를 순환시키고 문자 모드를 변경시키는 로직을 직접 작성하는데 어려움이 있었기 때문에 AI 의 도움을 받았다.

특히 다음 부분에서 AI 의 도움을 많이 받았다.

- 같은 버튼 연속 입력 처리
- 대소문자 전환
- 입력중인 문자 실시간 표시
- QTimer 를 이용한 문자 확정
- Enter 입력시 txt 파일에 문장추가
- 숫자 및 특수문자 모드
- 한글 자판 구현

이번 과제에서 코드작성시 AI 의 도움을 많이 받았고, UI 배치, 프로그램 실행 및 오류 발견은 직접 하였다.

2. AI 사용 부분



좋아. UI는 지금 충분히 잘 됐어. 이제 실제로 천지인식 영문 입력을 붙이자.

이번 단계에서 구현할 건 딱 이거야.

- ABC 연속 클릭 → A → B → C → A
- DEF, GHI 등도 동일
- 다른 버튼을 누르면 현재 작성 중인 글자를 확정하고 다음 글자 입력
- 약 1초 동안 추가 입력이 없으면 현재 글자 자동 확정
- Shift → 대문자/소문자 전환
- ← → 한 글자 삭제
- Space → 공백
- Enter → 작성 중인 문장을 output.txt 마지막 줄에 추가
- 입력 중인 글자도 inputLineEdit 에 실시간 표시

과제 필수조건인 대소문자, 작성 중 문자 실시간 출력, Enter 시 txt 파일 추가를 여기서 한 번에 처리하는 거야.

📄 CQt_day3

2.1 영문 문자 입력

하나의 버튼에 여러 문자를 넣고 반복해서 누르면 문자가 바뀌도록 하는 방법을 AI 에게 질문 하였다.

이를 위해 현재 어떤 버튼을 누르고 있는지 currentKey 에 저장하고, 현재 몇 번째 문자를 선택하고 있는지는 currentIndex 에 저장하도록 하였다.

같은 버튼을 누르면 currentIndex 를 증가시키고, 다른 버튼을 누르면 현재 문자를 확정된 뒤 새로운 문자 입력을 시작하도록 구현하였다.

2.2 QTimer 를 이용한 문자 확정

처음에는 같은 버튼을 여러번 누르는 방식에서 어떤 방식으로 언제 문자를 확정할 수 있게 만들지 어려웠다.

AI 의 도움을 받아 QTimer 를 사용하여 일정 시간 동안 추가 입력이 없으면 현재 문자를 자동으로 확정하도록 구현하였다.

예를 들어 ABC 버튼을 눌러 B 를 만든 뒤 일정 시간 동안 아무 버튼도 누르지 않으면 B 가 확정되는 방식이다.

3. 이 코드가 어떻게 동작하는지

예를 들어 ABC 를 누르면:

```
1회 → A
2회 → B
3회 → C
4회 → A
```



화면에서도 바로:

```
A
↓
B
↓
C
```



로 바뀐.

그리고 ABC 로 C 를 만든 상태에서 DEF 를 누르면:

```
C 확정
+
D 작성 중
```



이 돼서 화면에는:

```
CD
```



가 보임.

약 1초 동안 아무 버튼도 누르지 않아도 현재 문자가 자동 확정돼.

2.2 A* 및 Dijkstra 알고리즘 구현

A* 및 Dijkstra 알고리즘을 구현할때 AI 의 도움을 받았다. hw1 보고서를 작성하며 두개의 알고리즘의 개념은 알고있었지만, 실제 C++코드로 구현하려 하니 익숙치 않아어려움이 있었다. 따라서 AI 의 도움을 받아 Dijkstra 알고리즘은 각 노드까지의 최소 비용을 저장하고 priority_queue 를 이용하여 현재까지의 비용이 가장 적은 노드를 먼저 탐색하는 구조로 구현하였고, A*알고리즘 또한 AI 의 도움을 받아 $f(n) = g(n) + h(n)$ 을 이용하여 코드를 작성하였다.

좋아. 이제 Dijkstra부터 하나씩 붙이자.

이번 단계 목표는:

시작점/도착점/장애물을 만든 뒤 Apply 를 누르면 Dijkstra 쪽에서 탐색한 칸 + 최단 경로 + Cost / Visited / Time이 표시되게 만들기.

과제에서 요구하는 비교 항목도 최종 경로 비용 / 탐색한 노드 수 / 실행 시간 이야. CQt_day3

헛갈리지 않게 이번에도 mainwindow.h, mainwindow.cpp 전체 코드로 줄게.

1. mainwindow.h 전체

```
// C++

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QTableWidget>

#include <vector>
#include <utility>

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void on_createMapButton_clicked();

    void on_astarTable_cellClicked(int row, int column);
    void on_dijkstraTable_cellClicked(int row, int column);

    void on_applyButton_clicked();

private:
    Ui::MainWindow *ui;

    // 맵 데이터
    // 0 : 빈칸
    // 1 : 장애물
    // 2 : 시작점
    // 3 : 도착점
    std::vector<std::vector<int>> mapD;
```

좋아. 이제 왼쪽 A*까지 붙여서 A*와 Dijkstra가 같은 맵에서 동시에 탐색되는 형태로 만들자. PPT에서도 A*는 $f(n)=g(n)+h(n)$ 이고, Dijkstra는 $f(n)=g(n)$ 으로 비교하고 있어. CQt_day3

이번에 바뀌는 핵심은 3개야.

- runAStar() 추가
- Apply 누르면 A*와 Dijkstra 둘 다 계산
- 하나의 QTimer 로 양쪽 탐색을 동시에 한 칸씩 시각화

2.3 입력 중인 문자 실시간 출력

입력중인 문자도 실시간으로 출력하기 위하여 이미 확정된 문자열은 committedText 에 저장하고, 아직 확정되지 않은 문자는 따로 계산하여 QLineEdit 에 함께 표시하도록 구현하였다.

예를 들어 ABC 버튼을 반복해서 누르면 입력창에 있는 글자중 현재 입력중이라고 판단되는 문자가

$A \rightarrow B \rightarrow C$ 로 변경된다.

2.4 한글 자판 구현

한글 자판은 AI의 도움을 받아 천지인 방식의 모음과 여러 자음 버튼을 구성하였다.

한글은 영어처럼 단순히 문자를 이어 붙이는 방식이 아니라 초성, 중성, 종성을 서로 조합해야하기 때문에 별도의 상태를 관리하였다.

받침이 있는 상태에서 모음이 입력되면 기존 받침을 다음 글자의 초성으로 이동시키는 처리도 구현하였다.

The screenshot displays a Korean input interface with the following components:

- 가 입력** (ga input): The current input character.
- 순서:** (Order): A label for the input sequence.
- 결과:** (Result): A label for the final output.
- 나** (na): The next input character.
- 고** (go): The next input character.
- 구** (gu): The next input character.
- 받침** (받침): A label for the final output.
- 예를 들어:** (For example): A label for the example input.

The interface shows a sequence of input boxes for '가', '나', '고', and '구', each with a copy icon. Below these, there are input boxes for '가', '나', '고', and '구' with a copy icon. At the bottom, there is a '받침' (받침) section with an '예를 들어:' (예를 들어:) label and an input box containing '가' and a '↓' (down) button.

3. 깨달은 점

이번 과제를 통해 Qt에서 UI와 C++ 코드가 어떻게 연결되는지도 이전보다 더 이해할 수 있었으며 특히 QPushButton과 objectName을 기준으로 클릭 이벤트 함수가 연결되고, QLineEdit의 내용을 코드에서 변결할 수 있다는 것을 깨달았다. 또한 QTimer를 활용한 시간 기반 이벤트 처리방법도 알 수 있었던 것 같다.

이번 과제에서는 코드 작성 부분에서 AI의 도움을 많이 받았지만, UI를 직접 구성하고 프로그램을 반복해서 실행하면서 오류를 찾고 수정하는 과정을 통해 **AI가 생성한 코드의 동작을 확인하고 필요한 부분을 판단하는 것이 중요하다는 점**을 느꼈던 시간이었던 것 같다.

이번 과제에서는 UI 구성은 직접 진행하였고, 문자 입력 로직, 파일 입출력, 한글 조합 및 오류 수정 과정에서 AI의 도움을 많이 받았다. 하지만 AI의 도움을 받는 것에서 그치지 않고 코드를 이해하면서 Qt와 C++의 동작 원리를 알아가려고 노력하였으며, 각 함수의 역할도 직접 설명할 수 있도록 이해하려고 하였다. 앞으로는 코드 작성을 AI에게 맡기는 것에 의존하기보다, 스스로 주도적으로 코드를 작성할 수 있도록 노력할 것이다.